

Dans les séquences précédentes, vous avez déjà vu des algorithmes. Le but de cette séquence est de :

- ✚ Découvrir d'autres algorithmes (K plus proches voisins, Gloutons).
- ✚ D'utiliser des jeux de tests pour mettre au point ces programmes.
- ✚ D'utiliser la documentation des bibliothèques pour coder l'algorithme.
- ✚ D'importer un fichier csv.

1. Algorithme des K plus proches voisins :

C'est un algorithme de machine learning.

L'idée est d'utiliser un grand nombre de données afin "d'apprendre à la machine" à résoudre un certain type de problèmes.

Dis-moi qui sont tes voisins, je te dirai qui tu es.

Exemple : pour classer différents types d'IRIS :

Pour chaque iris étudié, Anderson a mesuré (en cm) :

- ✚ la largeur des sépales
- ✚ la longueur des sépales
- ✚ la largeur des pétales
- ✚ la longueur des pétales

A l'aide d'un fichier csv, on a :

- ✚ la longueur des pétales
- ✚ la largeur des pétales
- ✚ l'espèce de l'iris (au lieu d'utiliser les noms des espèces, on utilisera des chiffres : 0 pour "iris setosa", 1 pour "iris virginica" et 2 pour "iris versicolor")

	A	B	C
1	petal_length	petal_width	species
2	1.4	0.2	0
3	1.4	0.2	0
4	1.3	0.2	0
5	1.5	0.2	0
6	1.4	0.2	0
7	1.7	0.4	0
8	1.4	0.3	0
9	1.5	0.2	0
10	1.4	0.2	0
11	1.5	0.1	0
12	1.5	0.2	0
13	1.6	0.2	0
14	1.4	0.1	0
15	1.1	0.1	0
16	1.2	0.2	0
17	1.5	0.4	0
18	1.3	0.4	0
19	1.4	0.3	0
20	1.7	0.3	0
21	1.5	0.3	0

Pour cette partie, il faut commencer par récupérer le fichier [iris.csv](#)

Dans un premier temps, nous allons afficher un graphique qui en abscisse représente la longueur des pétales, en ordonnée la largeur des pétales et une couleur par espèce (vert pour setosa, rouge pour virginica et bleu pour versicolor) pour cela nous utiliserons la bibliothèque matplotlib.

Pour faire le graphique, il faut d'abord charger le fichier csv pour récupérer les données. Nous allons utiliser la bibliothèque pandas (import pandas).

I) Écrire un script avec comme code **import pandas**

Sauver sous KNN.py puis exécuter.

Dans la console, taper `help(pandas)`

Vous obtiendrez la documentation de la librairie. On voit que cette librairie a une fonction qui permet de lire un fichier.

Pour lire il suffit donc de coder :

iris=pandas.read_csv("iris.csv")

N'oubliez pas de mettre le fichier csv dans le même répertoire que le programme

On peut aussi voir pour la bibliothèque matplotlib.

II) Écrire à la suite **import matplotlib.pyplot as plt.**

Sauver sous KNN.py puis exécuter.

Dans la console, taper `help(plt)`

On constate que la documentation peut être très longue. Pour vous simplifier la tâche, nous allons utiliser les lignes de code suivantes :

```

Help on package pandas:

NAME
pandas

DESCRIPTION
pandas - a powerful data analysis and manipulation library for Python
=====

See http://pandas.pydata.org/ for full documentation. Otherwise, see the
docstrings of the various objects in the pandas namespace:

Series
DataFrame
Panel
Index
DatetimeIndex
HDFStore
bdate_range
date_range
read_csv
read_fwf
read_table
ols
  
```

```

x=iris.loc[:, "petal_length"]
y=iris.loc[:, "petal_width"]
lab=iris.loc[:, "species"]
plt.scatter(x[lab == 0], y[lab == 0], color='g', label='setosa')
plt.scatter(x[lab == 1], y[lab == 1], color='r', label='virginica')
plt.scatter(x[lab == 2], y[lab == 2], color='b', label='versicolor')
plt.legend()
plt.show()
  
```

III) Écrire à la suite la ligne de code pour lire le fichier csv puis les 8 instructions en gras ci-dessus. Vous commenterez précisément chaque ligne du code (comme l'exemple ci-dessous)

```
import pandas #importation de la bibliothèque pandas pour lire un fichier csv
import matplotlib.pyplot as plt #importation de la bibliothèque matplotlib pour faire un graphique
```

Sauver sous KNN.py puis exécuter.

Vous devriez obtenir cela :

Nous obtenons des "nuages" de points, on remarque ces points sont regroupés par espèces d'iris (sauf pour "iris virginica" et "iris versicolor", les points ont un peu tendance à se mélanger).

Au cours d'une promenade on trouve un iris, n'étant pas un spécialiste, il n'est pas vraiment possible de déterminer l'espèce. En revanche, on peut mesurer la longueur et la largeur des pétales de cet iris. Admettons qu'un pétale fasse 0,5 cm de large et 2 cm de long. Plaçons cette nouvelle donnée sur notre graphique (il nous suffit d'ajouter la ligne

plt.scatter(2.0, 0.5, color='k'), le nouveau point va apparaître en noir (color='k'))

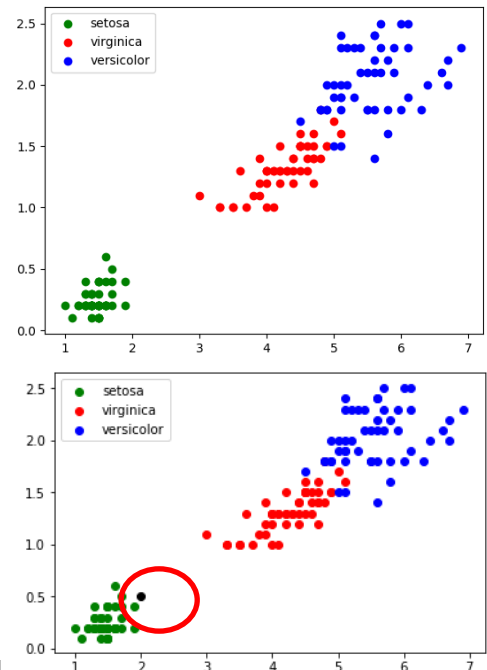
Le résultat est sans appel : il y a de fortes chances que votre iris soit de l'espèce "iris setosa".

Maintenant posons-nous la question avec une largeur du pétale = 0,75 cm et longueur du pétale = 2,5 cm.

IV) Rajouter la ligne de code qui permet d'afficher en noir le point (2.5,0.75).

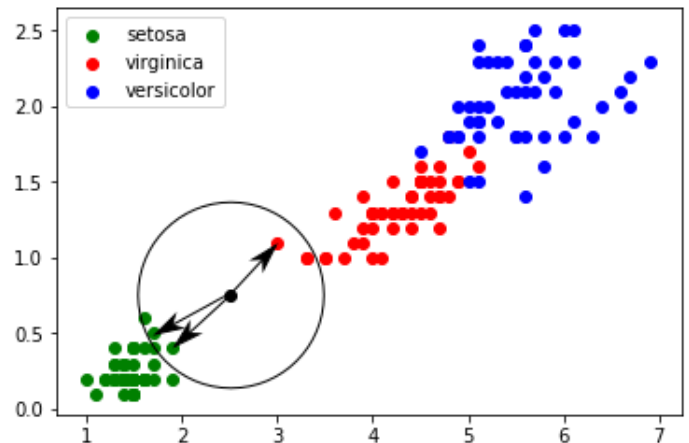
Pouvez-vous déterminer le type d'iris ?

Sauver sous KNN.py puis exécuter.



Pour déterminer l'espèce, nous allons utiliser l'algorithme des k plus proches voisins :

- ✚ On calcule la distance entre notre point (largeur du pétale = 0,75 cm ; longueur du pétale = 2,5 cm) et chaque point issu du jeu de données "iris" (à chaque fois c'est un calcul de distance entre 2 points tout ce qu'il y a de plus classique).
- ✚ On sélectionne uniquement les k distances les plus petites (les k plus proches voisins)
- ✚ Parmi les k plus proches voisins, on détermine quelle est l'espèce majoritaire. On associe à notre "iris mystère" cette "espèce majoritaire" parmi les k plus proches voisins



Si k=3 voici l'illustration, on trouverait iris-setosa.

Parmi tous ces algorithmes, la bibliothèque Scikit Learn propose l'algorithme des k plus proches voisins.

Après avoir importé la bibliothèque : **from sklearn.neighbors import KNeighborsClassifier**

L'algorithme peut être codé de la façon suivante :

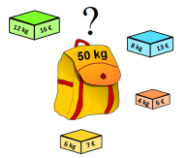
d=list(zip(x,y)) #permet de passer des 2 listes x et y à une liste de tuple d = [(1.4, 0.2), (1.4, 0.2),...]

model = KNeighborsClassifier(n_neighbors=k) #méthode issue de la bibliothèque scikit-learn qui prend le KNN où k est le nombre de voisins les plus proches

model.fit(d,lab) #associe les tuples présents dans la liste "d" avec les labels (0 : "iris setosa", 1 : "iris virginica" ou 2 : "iris versicolor")

prediction= model.predict([[longueur,largeur]]) #La variable "prediction" contient alors le label trouvé (0,1ou 2)par l'algorithme knn où longueur correspond à la longueur mystère et largeur à la largeur mystère.

Pour afficher les résultats sur le graphique, on utilisera cette partie de code :



```
#Affichage résultats
txt="Résultat : "
if prediction[0]==0:
    txt=txt+"setosa"
if prediction[0]==1:
    txt=txt+"virginica"
if prediction[0]==2:
    txt=txt+"versicolor"
plt.text(3,0.5,"largeur : {0} cm, longueur : {1} cm.".format(largeur,longueur),fontsize=12)
plt.text(3,0.3, "k : {0}".format(k), fontsize=12)
plt.text(3,0.1, txt, fontsize=12)
#fin affichage résultats
```

V) Rajouter la ligne de code qui permet d'importer la bibliothèque.

Rajouter trois lignes où l'on définit les variables :

longueur = 2.5

largeur = 0.75

k=3

Rajouter l'algorithme knn

Rajouter l'affichage des résultats

Sauver sous KNN.py puis exécuter.

Quel est l'iris mystère ?

VI) Modifier le programme pour avoir $k=5$.

Sauver sous KNN.py puis exécuter.

Quel est l'iris mystère ?

Conclusion

2. Algorithmes gloutons :

Le principe de tels algorithmes consiste à choisir des solutions locales d'un problème dans le but d'obtenir une solution globale au problème. Cette solution n'est pas forcément optimale.

a) Le rendu de monnaie :

Le rendu de monnaie : 12€03 parmi les pièces disponibles : 5€, 1€, 0.5€ et 0.01€



Une solution est construite pas à pas, en choisissant à chaque étape le meilleur élément localement. On commencera par les plus grandes valeurs et on regardera combien de fois elles peuvent être utilisées au maximum.

On choisira 2 billets de 5 euros puis 2 pièces de 1 euros et 3 pièces de 1 cents.

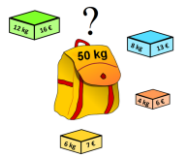
Les pièces de monnaies ont été choisies de telle sorte que cet algorithme soit optimal c'est pour cela qu'il n'y a pas de pièces de 4 euros. Elles ont été choisies de façon canonique.

Le but de la question suivante est de coder cet algorithme.

VII) Commencer par créer deux variables :

- + **pièces** qui sera un tableau qui contient les valeurs des pièces et billets en cents soit [500,100,50,1].

 **somme_a_rendre** qui sera un float donner par l'utilisateur grâce à la fonction input.



Ensuite créer la fonction : **def rendu (somme_a_rendre) :**

- ✚ On créera une assertion qui affichera « On ne peut pas rendre une somme négative » dans la console si l'utilisateur donne une somme ≤ 0 .
- ✚ Si la somme est ≥ 0 , on créera un tableau vide rendu qui contiendra la quantité de chaque pièce à rendre. Ce tableau sera rempli grâce à une boucle for en utilisant la méthode `rendu.append(.....)` puis on créera une deuxième boucle for qui va afficher « Il faut rendre pièces de euros »

Pour tester le programme, on ajoutera la ligne de code **rendu(somme_a_rendre*100) #on multiplie par 100 la somme à rendre car le tableau de pièces est en cents.**

Sauver sous `monnaie.py` puis tester pour somme à rendre = 12.03€.

VIII) Modifier le programme pour avoir toutes les pièces en euros (2€, 1€, 0.5€, 0.2€, 0.1€, 0.05€ et 0.01€.

Sauver sous `monnaie.py` puis tester pour somme à rendre = 12.03€, 5.14€ et 55 cents.

Voyons s'il est toujours optimal. Le but est de construire un mur en Lego qui fait 12 plots de longueur sur 2 plots de largeur.

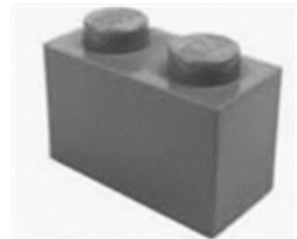
Nous avons à disposition des Legos suivant :



5*2



4*2



1*2

IX) Modifier le programme pour avoir les pièces 5,4,1 et remplacer `somme_a_rendre` par `mur`
Sauver sous `lego.py` puis tester pour `mur = 12`.

```
*** Console de processus distant Réinitialisée ***
On ne peut pas construire un mur d'une longueur négative
>>>
*** Console de processus distant Réinitialisée ***
Il faut prendre 2.0 lego(s) de 5 plots
Il faut prendre 0.0 lego(s) de 4 plots
Il faut prendre 2.0 lego(s) de 1 plots
>>>
```

Avec cet algorithme on trouve un total de 4 lego en tout.

X) Est-ce la solution optimale (c'est-à-dire la solution qui utilise le moins de Lego) ?

b) Le sac à dos :

On dispose d'un sac pouvant contenir un poids maximal P
On dispose par ailleurs d'un ensemble de n objets ayant chacun un poids ainsi qu'une valeur.

Question : Quels objets prendre afin d'emporter la plus grande valeur possible, tout en ne dépassant pas le poids que peut contenir le sac ?

Exemple :

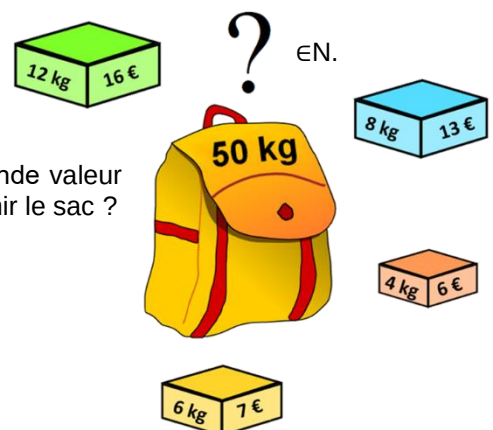
Poids maximal du sac $P=50\text{kg}$

Objet 1 : $O1 = (4, 6)$ (poids 4kg et valeur 6€)

Objet 2 : $O2 = (6, 7)$

Objet 3 : $O3 = (8, 13)$

Objet 4 : $O4 = (12, 16)$

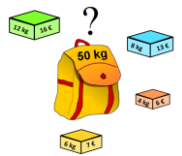


Si on prend 4 objets $O4$ on aura un poids de 48kg et une valeur de 64€

Si on prend 1 objet $O1$ et 1 objet $O2$ et 2 objets $O3$ et 2 objets $O4$ on aura :

✚ Un poids de $1*4 + 1*6 + 2*8 + 2*12 = 50\text{kg}$

✚ Une valeur de $1*6 + 1*7 + 2*13 + 2*16 = 71\text{€}$



Si on prend 3 objets O3 et 2 objets O4, on aura :

- ✚ Un poids de $2*8 + 2*12 = 48\text{kg}$
- ✚ Une valeur de $3*13 + 2*16 = 71\text{€}$ même valeur mais poids plus léger.

Comment trouver la réponse convenable au problème posé ?

- ✚ On va créer un tableau qui sera le rapport de la valeur sur le poids de chaque objet
- ✚ On tri par insertion ce tableau en ordre croissant (à récupérer de la séance S4 Exo13.py) puis on fera un `tab.reverse()` pour l'avoir dans l'ordre décroissant juste avant de faire `return`.
- ✚ On applique le principe de l'algorithme glouton : mettre le maximum à chaque étape des objets triés.

XI) Créer un tableau deux dimensions objets contenant les valeurs : `[[4,6],[6,7],[8,13],[12,16]]`. Recopier la fonction `triInsertion` de la séance S4 et modifier cette fonction pour avoir l'ordre décroissant.

Créer la fonction `sac_a_dos(objets,poids_max)`:

- ✚ Créer en premier le tableau rapport qui contient le quotient de la valeur par le poids pour chaque objet.
- ✚ Copier par compréhension ce tableau dans un tableau qui se nomme `ordre`. (une fois trié il faudra savoir à quel objet correspond chaque rapport)
- ✚ Trier le tableau rapport
- ✚ Créer in tableau indices qui correspond à l'ordre des objets du nouveau tableau rapport. Pour cela, commenter la partie de code suivante et insérer la dans votre script :

```
for i in range(0,len(rapport)):
    for j in range(0,len(rapport)):
        if rapport[i]==ordre[j]:
            indices.append(j)
```

- ✚ Coder l'algorithme glouton en complétant le tableau `a_prendre` qui contiendra les quantités de chaque objet à prendre.
- ✚ Afficher le résultat sous la forme : Il faut prendre objet(s)
- ✚ Calculer le `poids_total` et la `valeur_total`
- ✚ Afficher Le poids total sera de : Kg . La valeur du sac est de :€

Demander à l'utilisateur de donner le poids maximum souhaité

Appeler la fonction `sac_a_dos(objets,poids_max)`

Sauver votre programme sous `sac_a_dos.py` et tester avec poids maxi = 50kg. Vous devez obtenir :

```
Il faut prendre 6 objet(s) 3
Il faut prendre 0 objet(s) 1
Il faut prendre 0 objet(s) 4
Il faut prendre 0 objet(s) 2
Le poids total sera de : 48 kg
La valeur du sac est de :78 €
```

Utilisons cet algorithme dans un cas plus réel. Un cambrioleur rentre dans une maison et souhaite que son butin ait la plus grande valeur. Il estime qu'il a 4 objets qui ont une valeur non négligeable :

Objet 1 : O1= (13,70) (poids 13kg et valeur 70€)

Objet 2 : O2= (12,40)

Objet 3 : O3= (8,30)

Objet 4 : O4= (10,30)

Il ne pourra porter que 30 kilos.

XII) Modifier votre programme afin qu'on ne puisse que prendre 1 objet de chaque.

Affecter les valeurs suivantes à la variable « objets »:
`[[13,70],[12,40],[8,30],[10,30]]`.

Sauver sous `cambrioleur.py` et tester avec poids maxi=30kg. Qu'obtenez-vous ?

Quel est le poids et la valeur du butin s'il avait pris un objet1 et un objet2 ?

La solution trouvée n'est donc pas optimale mais reste rapide même si le nombre d'objets augmente considérablement. La méthode exacte consisterait à énumérer intelligemment toutes les solutions.

En pratique, seules les solutions potentiellement de bonne qualité seront énumérées, les solutions ne pouvant pas conduire à améliorer la solution courante ne sont pas explorées.

Pour déterminer la solution, il suffit de calculer la valeur du sac pour chaque nœud et de prendre la solution ayant la plus grande valeur et le poids minimum.

